



Generación autónoma de microservicios

Dónde está la industria · cómo funciona · qué cuesta · hacia dónde vamos

Dónde está la industria.

1 · Agentes generalistas

- Arreglan o completan código **ya existente**
- Dominan los rankings
- Saturados y **caros** (\$5–\$30 por tarea)

2 · Generadores guiados por especificación

- Construyen **desde una especificación**
- **Nuestro camino**
- Solo 12–55% resuelto: mucho espacio

3 · Agentes que se auto-mejoran

- Escriben su propio «manual»
- Prometedor en laboratorio
- En código real: mejora **casi nula**

La lección de toda la investigación: el ingrediente que gana es una **verificación de fiar**, no un modelo más listo.

El dolor: del síntoma a la causa raíz.

EL PROBLEMA DE FONDO

Microservicios caros y con deuda

Generar un servicio productivo exige capas estrictas, contratos inmutables, gestión centralizada de errores y compilación hermética.

Cualquier desliz se paga en semanas de refactor o defectos en producción.

EL SÍNTOMA REAL

«0 de 15 sesiones verificadas»

Durante meses la plataforma generaba código pero **no podía demostrar que su propia salida compilaba**.

La causa: un defecto en el volumen Docker de **un solo carácter** que bloqueaba la verificación.

EL APRENDIZAJE CLAVE

Sin verificación no hay fiabilidad

Un sistema que no compila y prueba herméticamente su código no puede prometer gobernanza.

La solución: nadie da por bueno un servicio hasta que mvn test -o pasa en un contenedor aislado.

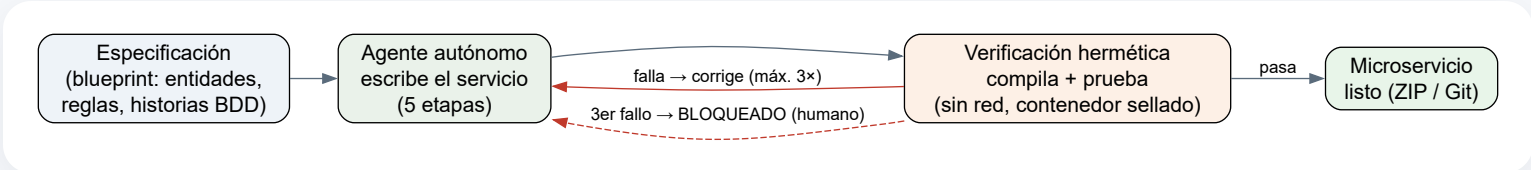
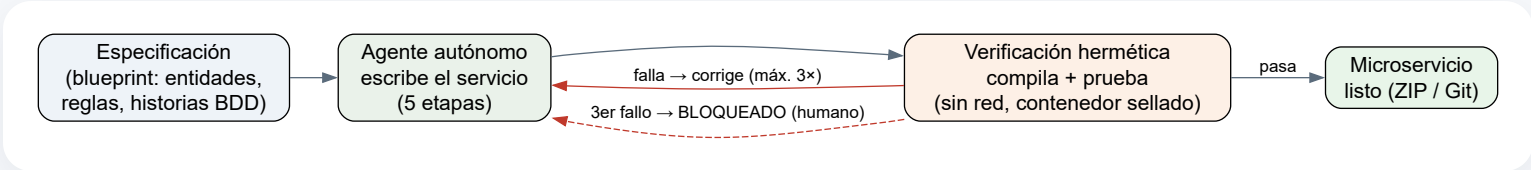
La promesa de AgentIA no es «escribir código más rápido»: es **gobernanza verificada, auditable y hermética**.

Spec-Driven Design: la especificación manda.

Artefacto de Especificación	Qué prescribe y garantiza formalmente
Blueprint / spec.md (Spec Kit)	Entidades, tipos de datos, relaciones JPA, validaciones de dominio e historias BDD.
Constitución (6 principios)	Capas limpias, inmutabilidad de contratos, errores unificados, modo offline sin red, quality gates y protección de secretos.
Instrucciones de etapa (5 docs)	Procedimiento formal paso a paso para cada agente (sin improvisación).
Lista blanca de dependencias	Límite estricto de dependencias en pom.xml (cero dependencias huérfanas o no autorizadas).
Contratos de integración (19)	Definición canónica de comportamiento e interfaces entre subsistemas.

La especificación es la autoridad, no el modelo: el LLM actúa como compilador de la especificación a código Java ejecutable.

Cómo funciona la solución.



Orquestación LangGraph: 8 agentes coordinados.

Validador → Scaffolder → Dominio → Servicio → Controlador → Test → Sandbox Docker ⇄ Reparación

5 AGENTES ESPECIALIZADOS LLM

Especialistas de Dominio Técnico

- **1. Scaffolder:** Estructura Maven y dependencias de lista blanca.
- **2. Dominio:** Entidades JPA, records inmutables, repositorios.
- **3. Servicio:** Lógica transaccional y reglas de negocio.
- **4. Controlador:** Endpoints REST y `@RestControllerAdvice`.
- **5. Test:** Cobertura de integración y unitaria mapeada a BDD.

3 AGENTES DETERMINISTAS (ZERO-LLM)

Gobernanza y Verificación Hermética

- **6. Validador estático:** Analiza sintaxis y esquemas JSON antes de cualquier compilación.
- **7. Sandbox Docker:** Ejecuta `mvn test -o` en contenedor hermético sin acceso a red.
- **8. Reparación reflexiva:** Analiza traza de error y reintenta con límite estricto de **máx. 3 iteraciones** antes de escalar a humano (BLOQUEADO).

Anatomía del prompt y reglas de ingeniería.

Precedencia estricta de inyección

- **1. Prefijo de habilidad (SkillOpt):** directrices procedimentales aprendidas.
- **2. System Prompt:** documento de instrucción inmutable por rol.
- **3. Payload de la tarea:** blueprint de la feature y contexto de etapas previas.
- **4. Salida estructurada:** esquema JSON estricto sin preámbulos conversacionales.

Reglas de arquitectura forzadas

Dominio: records nativos de Java para DTOs; @Data de Lombok estrictamente prohibido.

Servicio: inyección de dependencias exclusivamente por constructor; transaccionalidad explícita.

Controlador & Tests: @RestControllerAdvice centralizado con traza RFC-7807; cada escenario BDD tiene un @Test asociado.

El proceso, de punta a punta.



Nueve etapas, de la especificación a la entrega — con la generación y la verificación en el centro.

Lo que eso garantiza.

- **Se escribe** un servicio completo, no fragmentos: entidades, lógica, API, tests.
- **Se verifica** en un contenedor sellado, sin red: compila y pasa sus pruebas, o no se entrega.
- **Se autocorriga** hasta 3 veces; al tercer fallo se detiene y pide a un humano.
- **Se audita** el cumplimiento de las reglas del proyecto — no la opinión del modelo.

La interfaz de usuario: React/Vite y 10 pestañas.

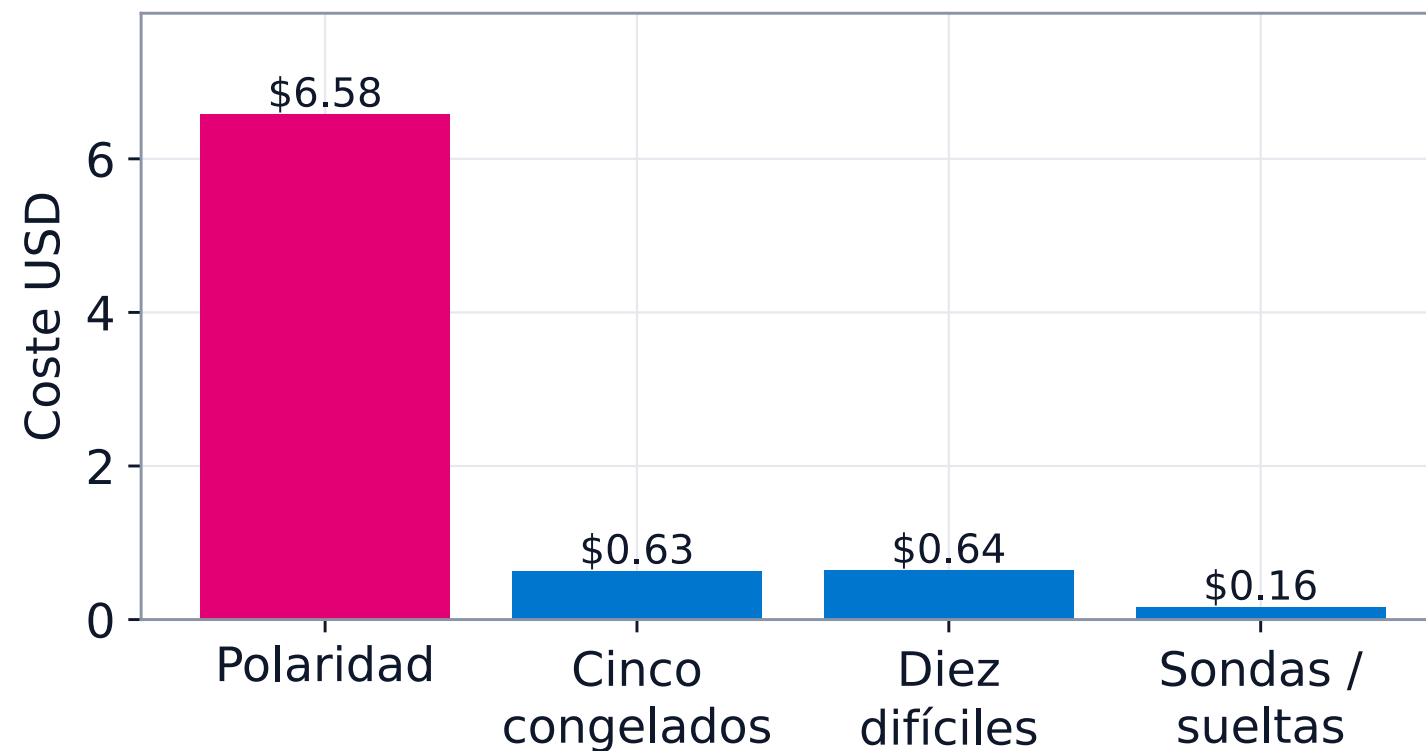
SPA moderna con 10 fases: Requisitos, Arquitectura, Modelos, Blueprints, Generación, Código/Fix, Calidad SAST, DevOps, Entrega y Configuración.

Streaming SSE en vivo: Pestaña 5 con consola en tiempo real, eventos estructurados por nodo LangGraph y métricas de latencia por agente.

Visores Mermaid nativos: renderizado en cliente de diagramas de componentes y diagramas ER generados por los agentes.

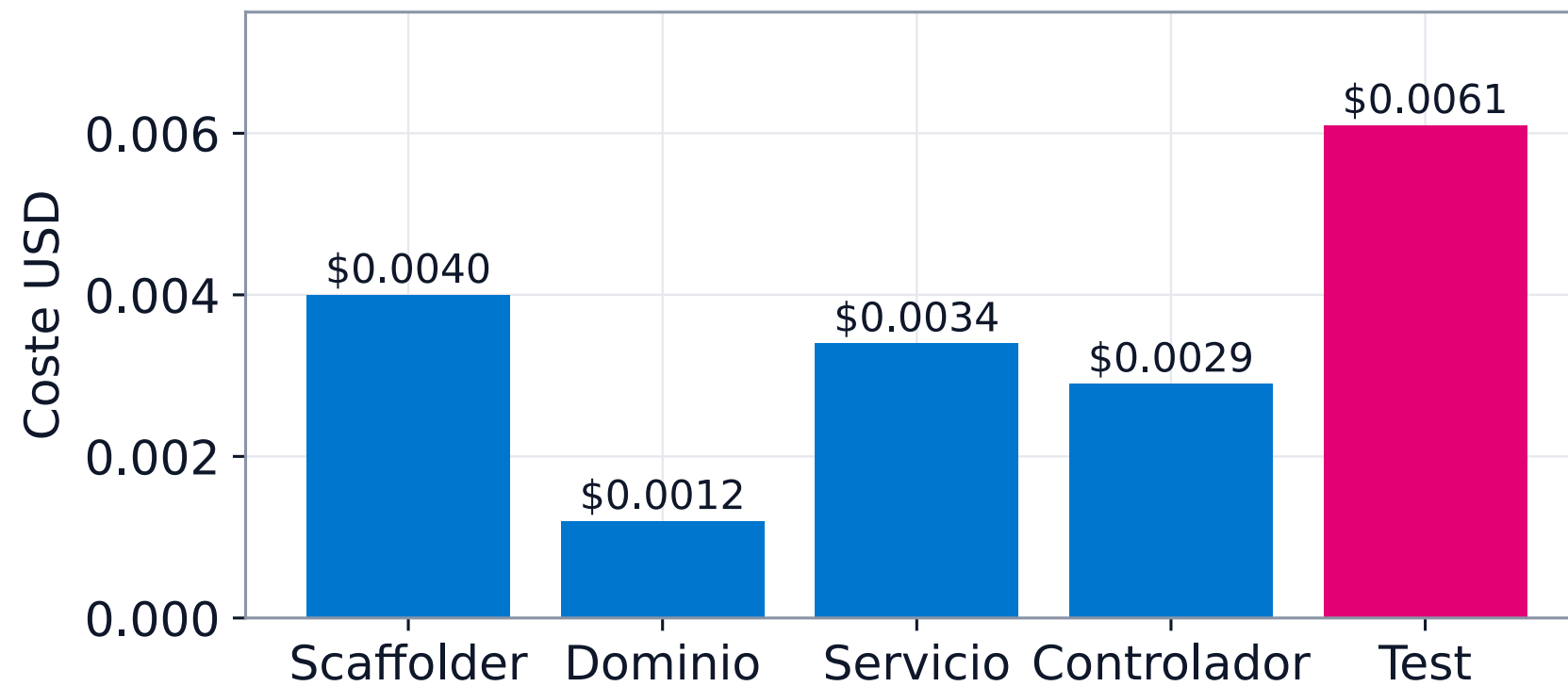
Soporte Multi-LLM verificado: selector dinámico para Gemini, Groq, OpenAI, DeepSeek y mock determinista, con verificación de 3 estados sin alucinaciones.

El coste.



- Un servicio (dificultad media, recién ejecutado): **\$0.067**
- Rango por servicio: \$0.052–\$0.077
- Un servicio **garantizado** (3 candidatos, se entrega el que compila):
\$0.20
- Acumulado: **\$8.15** en 1.219 llamadas

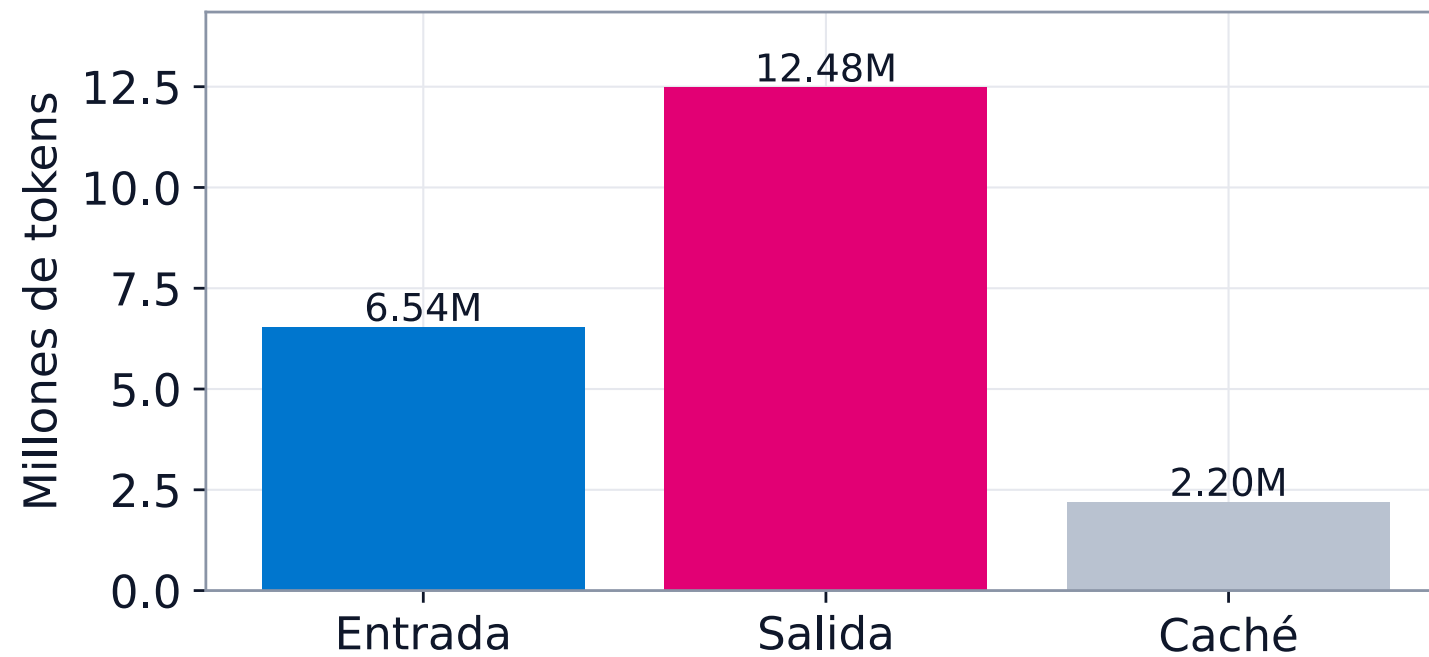
Coste por etapa.



El test es la etapa más cara (\$0.0061); dominio la más barata (\$0.0012). El coste escala de forma predecible.

Tokens

(último reporte de MLflow) ●



19.0M tokens combinados

6.5M entrada · 12.5M salida

2.2M desde caché

- **78K tokens** por servicio
- 15.6K por llamada; salida $\approx 1.9\times$ la entrada

Estado actual: de la teoría a la verificación real.

- **Primera compilación verificada en sandbox hermético:** corrección del defecto de montaje Docker; la plataforma ejecuta `mvn test -o sin red` y valida compilación y pruebas BDD reales.
- **Paridad absoluta entre rutas de producto:** tanto la ruta secuencial como la orquestada por LangGraph compilan, ejecutan pruebas y comparten la misma veta de verificación.
- **Métrica de calidad inmanipulable y auditable:** transición de densidad heurística a penalización absoluta contra línea base congelada; auditoría eliminó resultados fabricados.
- **Registro honesto de costes en MLflow:** seguimiento riguroso de tokens de entrada, salida y caché; identificada discrepancia del 20% para corrección en SQLite.
- **Punto abierto identificado (33% intermitencia bajo carga):** 4/6 bajo concurrencia, 8/8 secuencial; atribuido a reetiquetado SELinux :Z en volumen compartido (en resolución).

Auto-mejora: el enfoque SkillOpt.

- **Fallo procedimental vs brecha de capacidad:** el LLM domina la sintaxis Java pero omite directivas de diseño; SkillOpt ataca la brecha de aplicación (*application gap*), elevando la fiabilidad en el dominio.
- **Qué es realmente una Habilidad (Skill):** no es un resumen informal, sino una política procedimental estricta con tres partes: **Trigger** (*when_to_apply*), **Procedimiento** (reglas ordenadas) y **Granularidad**.
- **Parámetro entrenable en el espacio de texto:** el documento de habilidad (300–2.000 tokens) se optimiza con rigor matemático sin reentrenar pesos neuronales ni incurrir en costes de GPU.
- **La condición crítica del oráculo:** «*un bucle sin diagnóstico detallado no supera a no tener bucle*» (CoEvoSkills: 41.1 con oráculo opaco vs 71.1 con verificador diagnóstico); por eso se ancla a trazas reales de `mvn test`.

Los 4 controles de fiabilidad en SkillOpt.

- **Presupuesto acotado de edición ($\leq L_t = 4$):** decaimiento coseno de ediciones permitidas por paso; previene reescrituras descontroladas y alucinaciones en el manual.
- **Compuerta de validación en datos retenidos (Held-out Gate):** una propuesta de regla se acepta *si y solo si* mejora estrictamente la puntuación en tareas no vistas; empates son rechazados.
- **Búfer de ediciones rechazadas:** las propuestas fallidas se realimentan al optimizador como contraejemplos negativos, evitando bucles y repetición de ideas descartadas.
- **Actualización lenta y meta-habilidad por época:** visión longitudinal que consolida aprendizajes globales; su ablación en SpreadsheetBench provocó una caída crítica de $\Delta = -22.5$ pts (de 77.5 a 55.0).

El camino hacia la auto-mejora gobernada.

Paso	Objetivo técnico	Esfuerzo / Coste
1	Verificador determinista: montaje hermético y resolución de reetiquetado SELinux	Corto (1–2 días)
2	Canal de diagnóstico rico: minería de fallos en reglas y extracción estructurada de trazas	Corto
3	Inyección y medición baseline: evaluar impacto de la habilidad Java/Maven inicial	\$3–\$5 en inferencia
4	Activación bucle SkillOpt: optimización reflexiva cerrada bajo los 4 controles de fiabilidad	Condicionado a pasos 1-3

De «no podía verificar su propia salida» a «**genera, compila y verifica por \$0.07, con ruta probada hacia la auto-mejora gobernada**».